Implement the Selection Sort and analyze the algorithm with different inputs

Selection Sort is a simple comparison-based sorting algorithm. Its primary idea is to divide the input list into two parts: a sorted sublist and an unsorted sublist. Initially, the sorted sublist is empty, and the unsorted sublist contains all the elements. The algorithm repeatedly selects the smallest (or largest, depending on the sorting order) element from the unsorted sublist and moves it to the end of the sorted sublist. This process continues until the unsorted sublist is empty and the sorted sublist contains all the elements in the desired order.

Algorithm of Selection Sort

Step 1 − Set MIN to location 0

Step 2 − Search the minimum element in the list

Step 3 − Swap with value at location MIN

Step 4 − Increment MIN to point to next element

Step 5 − Repeat until list is sorted

Basic concept and working principle

Let's see the basic working and principles of the selection sort algorithm using your example array: arr[] = {17, 34, 25, 49, 09}

Selection Sort Working Principle:

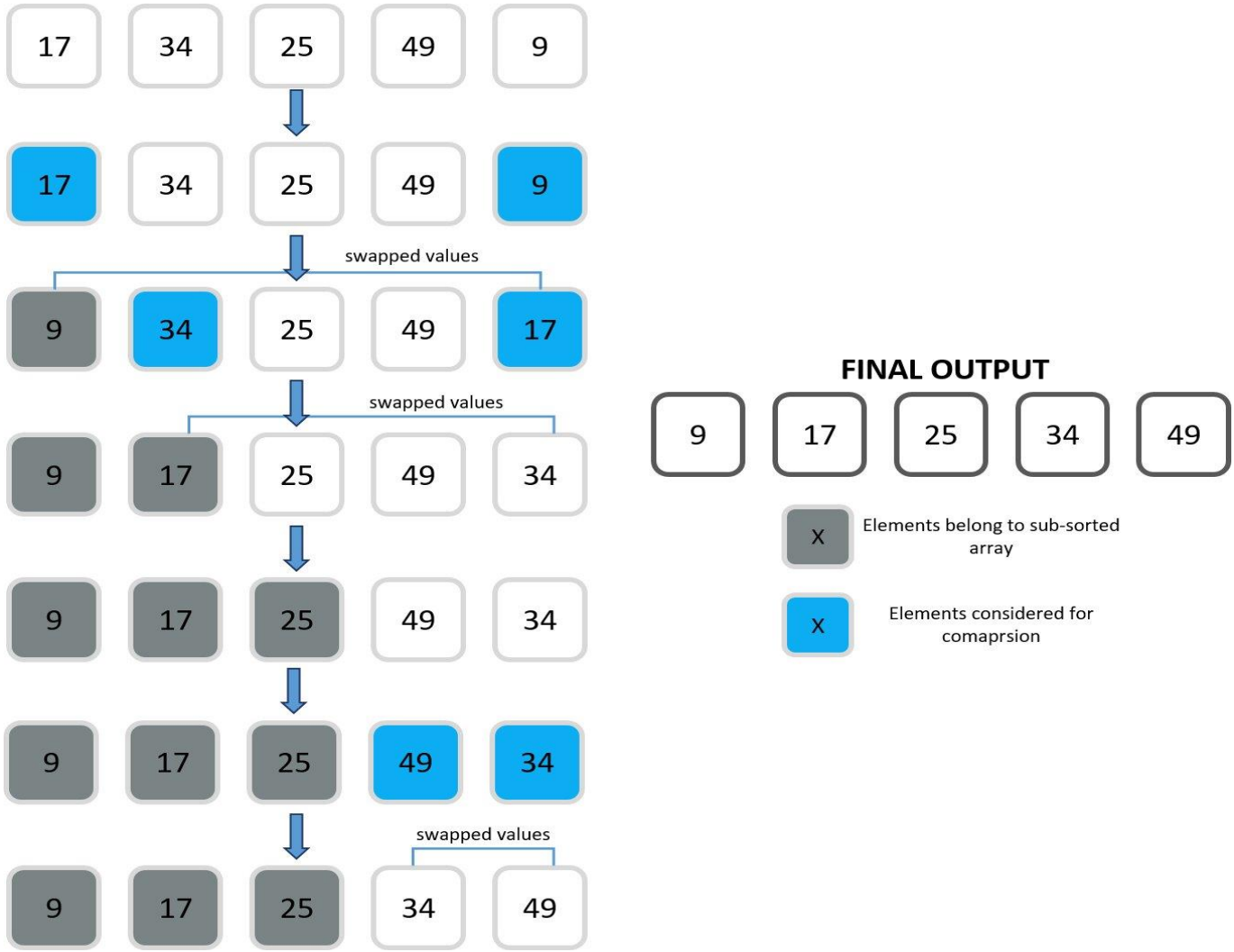
Initialization: Start with the first element as the current "minimum" (initially assuming it's the smallest).

Search for Minimum: Iterate through the remaining unsorted elements, comparing each one with the current "minimum." If you find a smaller element, update the current "minimum."

Swap: After completing the iteration and finding the actual minimum among the unsorted elements, swap it with the first unsorted element (the one you started with).

Repeat: Repeat steps 1-3 for the next unsorted element, then the next, until the entire array is sorted.

Here's how selection sort works step-by-step for your array



Pseudocode

Let's consider this pseudocode snippet

```
procedure selectionSort(arr: array of integer, n: integer)
   declare i, j, minIndex, temp: integer
   for i = 0 to n - 1 do
      minIndex = i
      for j = i + 1 to n - 1 do
         if arr[j] < arr[minIndex] then
            minIndex = j
         end if
      end for
      temp = arr[i]
      arr[i] = arr[minIndex]
      arr[minIndex] = temp
   end for
end procedure
```

Now, from the above pseudocode, let's understand the algorithm step by step.

Step 0: Consider the Initial Array

arr[] = {17, 34, 25, 49, 09}

The entire array is currently unsorted. The sorted portion is empty.

Step 1: 1st Iteration

Find the smallest element in the entire array and swap it with the first position.

Dry run

i	j	minIndex	arr[]	Comparison (arr[i] vs arr[j])	Swap?	Updated Array
0	1	0	[17, 34, 25, 49, 9]	17 < 34	No	[17, 34, 25, 49, 9]
0	2	0	[17, 34, 25, 49, 9]	17 < 25	No	[17, 34, 25, 49, 9]
0	3	0	[17, 34, 25, 49, 9]	17 < 49	No	[17, 34, 25, 49, 9]
0	4	4	[17, 34, 25, 49, 9]	17 > 9	Yes	[9, 34, 25, 49, 17]

arr[] = {09, 34, 25, 49, 17}

The first element, 09 is now in its correct position, and the array is divided into a sorted portion {09} and an unsorted portion {34, 25, 49, 17}.

Step 2: 2nd Iteration

Find the smallest element in the remaining unsorted portion {34, 25, 49, 17} and swap it with the second position.

Dry run

i	j	minIndex	arr[]	Comparison (arr[i] vs arr[j])	Swap?	Updated Array
1	2	1	[9, 34, 25, 49, 17]	34 > 25	No	[9, 34, 25, 49, 17]
1	3	1	[9, 34, 25, 49, 17]	34 < 49	No	[9, 34, 25, 49, 17]
1	4	4	[9, 34, 25, 49, 17]	34 > 17	Yes	[9, 17, 25, 49, 34]

arr[] = {09, 17, 25, 49, 34}

Here, sorted portion is {09,17} and unsorted portion is {25,49,34}

Step 3: 3rd Iteration

Find the smallest element in the remaining unsorted portion {25, 49, 34} and swap it with the third position.

Dry run

i	j	minIndex	arr[]	Comparison (arr[i] vs arr[j])	Swap?	Updated Array
2	3	2	[9, 17, 25, 49, 34]	25 < 49	No	[9, 17, 25, 49, 34]
2	4	2	[9, 17, 25, 49, 34]	25 < 34	No	[9, 17, 25, 49, 34]

At the end of this iteration, element 25 is already in its correct position.

arr[] = {09, 17, 25, 49, 34}

Here, sorted portion is {09,17,25} and unsorted portion is {49,34}

Step 4: 4th Iteration

Find the smallest element in the remaining unsorted portion {49, 34} and swap it with the fourth position.

Dry run

i	j	minIndex	arr[]	Comparison (arr[i] vs arr[j])	Swap?	Updated Array
3	4	3	[9, 17, 25, 49, 34]	49 > 34	Yes	[9, 17, 25, 34, 49]

Now, after the 4th iteration, only one element remains 49, which is inherently in its correct place as it's the last and largest element in the list.

So, at the end of the 4th iteration, the entire array is sorted in ascending order:

arr[] = {09, 17, 25, 34, 49}

That completes the selection sort for the given array.

Implementation of Selection Sort in C

Step 1: Function Declaration

The selection Sort function is declared to sort a given array.

Step 2: Outer Loop

This loop (for (i = 0; i < n – 1; i++)) traverses each element of the array, treating the current element as the boundary between the "sorted" and "unsorted" portions of the array.

The n-1 concept for the outer loop in the implementation of the Selection Sort algorithm has to do with the number of passes/iterations required to sort the array which in this case is 5-1 = 4 iterations (where, n=5)

Step 3: Assume Minimum

Within the outer loop, initially assume that the current element (i-th element) is the smallest (minIndex = i).

Step 4: Inner Loop for Minimum Search

The inner loop (for (j = i + 1; j < n; j++)) searches for the smallest element in the unsorted portion of the array. If a smaller element is found than the assumed minimum, the minIndex is updated.

Step 5: Swap

After the inner loop completes for each pass of the outer loop, the element at the minIndex (smallest in the unsorted portion) is swapped with the first element of the unsorted portion (i-th element).

Step 6: Repeat

The process continues until the outer loop has traversed the entire array, at which point the array will be sorted.

Step 7: Main Function

The main function initializes the array and its size, then calls the selectionSort function to sort the array. Before and after the sorting process, the array is printed to the console for visualization.

C Program for Selection Sort:

```
#include <stdio.h>
void selectionSort(int arr[], int n)
{
   int i, j;          // Outer, Inner loop counters
   int minIndex;      // Smallest element index
   int temp;          // Swap variable
   // Outer loop: up to second-last element
   for (i = 0; i < n - 1; i++)
         {
      minIndex = i;  // Start with current index
      // Inner loop: find smallest in unsorted part
      for (j = i + 1; j < n; j++)
         {
         if (arr[j] < arr[minIndex])
         {
            minIndex = j;  // Update smallest index
         }
      }
      // Swap
      temp = arr[i];
      arr[i] = arr[minIndex];
      arr[minIndex] = temp;
   }
}
int main()
{
   int arr[] = {17, 34, 25, 49, 9};   // Initial array
   int n = sizeof(arr) / sizeof(arr[0]);  // Array length
   // Display original array
   printf("Original Array: ");
   for (int i = 0; i < n; i++)
{
      printf("%d ", arr[i]);
   }
   printf("\n");
   selectionSort(arr, n);  // Sort the array
   // Display sorted array
   printf("Sorted Array: ");
   for (int i = 0; i < n; i++)
{
      printf("%d ", arr[i]);
   }
   printf("\n");
}
```